

Test Task - Conditional Diffusion (Cross-modal generation)

Context

This test task is small and self-contained: you train a conditional DDPM that generates images from a fixed-dimensional vector that was deterministically derived from a different "view" of the same sample. The goal is not just to make a model that works, but to think through how generation from a conditioning signal should be evaluated when there is a known mapping back from the generated output to that signal.

In the attachment you will find a small project skeleton: a fixed extractor that produces a 16-dimensional conditioning vector from each CIFAR-10 image (`pseudo_crossmodal.py`), a dataset preparation script that builds the paired (image, vector) dataset (`prepare_dataset.py`), and a small classifier you can use as a quality probe (`quality_probe.py`). Run `python prepare_dataset.py` once and `python quality_probe.py` once before starting.

Tasks

1. Using PyTorch, implement a conditional DDPM that generates 32×32 RGB images from the 16-dim conditioning vector. The diffusion code - noise schedule, forward process, training loss - must be written by you, not imported from `diffusers`, `denoising-diffusion-pytorch`, or similar libraries. Standard PyTorch layers (`nn.Conv2d`, `nn.GroupNorm`, attention, etc.) are of course fine. The choice of conditioning mechanism is yours; please justify it. A small-to-medium UNet (~3–10M parameters) is sufficient and trains in well under an hour on a single consumer GPU (Colab T4 is fine). Plot the training loss and any validation diagnostics you find useful, and comment on convergence.
2. Implement two samplers from scratch and apply them to the same trained model: ancestral DDPM sampling, and DDIM sampling. The two samplers must share the model - only the inference procedure changes between them. A bonus is a clean explanation of why DDIM is deterministic when $\eta=0$ and what that property implies about the relationship between a conditioning vector $\mathbf{c}$ and the generated image $\hat{\mathbf{x}}$ produced from it.
3. Design and implement a cycle-consistency-style evaluation. For a held-out set of conditioning vectors $\mathbf{c}$, generate images $\hat{\mathbf{x}}$ with your model, recompute the conditioning vector $\hat{\mathbf{c}}$ from $\hat{\mathbf{x}}$ using the provided extractor, and analyse the distribution of $\|\mathbf{c} - \hat{\mathbf{c}}\|$ together with a per-feature breakdown. The analysis is the deliverable here, not the number - try to understand which features of the conditioning vector your model preserves well and which it does not, and why.

4. Produce a quality/steps Pareto comparison between DDPM-ancestral and DDIM samplers at {500, 100, 50, 20, 10} steps. Use two evaluations: (a) the cycle-consistency metric you designed in task 3, and (b) the conditional accuracy of the provided classifier probe (sample from a known label distribution, classify, report agreement). Discuss the trade-offs you observe.

In all parts, the most important thing is to describe your reasoning and the motivation for the decisions you make as you make them - what you tried, what worked, what didn't, and what you would do with more time or compute. Negative results and dead ends are welcome and count in your favour. Format the result as one or several Jupyter notebooks committed to a git repository, with a short README describing how to reproduce. There is no length limit on your prose - a notebook with thoughtful markdown cells throughout is much more valuable to us than a short, polished notebook that hides the thinking.

Practical notes

If something is taking unreasonably long, write down what you would do next and stop; we'd rather see honest scope than a heroic all-nighter. Compute: a single consumer GPU (Colab T4 / 1080-class / M-series with MPS) is enough; we explicitly do not expect multi-GPU runs.